

THE BIBLE OF VIBE CODING

바이브 코딩

완전 복습 가이드

Gemini 래퍼 SaaS 프로젝트

비개발자를 위한 실전 바이블

NEXT.JS · SUPABASE · GEMINI · POLAR · VERCEL

PROLOGUE

Gemini 래퍼 SaaS 프로젝트 — 따라 해본 후 돌아보며 완전히 이해하기

이 문서는 코딩을 처음 접한 사람도 읽을 수 있도록 작성했습니다. 용어가 나올 때마다 일상 비유와 그림으로 설명하고, 전체 흐름은 다이어그램으로 한눈에 볼 수 있게 정리했습니다.

목차

0장. 코딩 용어 맛보기

- 0-1. 클라이언트 vs 서버 — 가장 중요한 구분
- 0-2. API — 프로그램끼리 대화하는 방법
- 0-3. URL, 도메인, 라우트
- 0-4. 환경변수 — 비밀번호 보관함
- 0-5. 데이터베이스(DB) — 정보를 저장하는 엑셀
- 0-6. 프론트엔드 vs 백엔드
- 0-7. 이제 본문으로

1장. 바이브 코딩이란?

- 1-1. 핵심 철학
- 1-2. 바이브 코딩의 3원칙
- 1-3. 내가 할 일 vs AI가 할 일

2장. 전체 시스템 한눈에 보기

- 2-1. 완성된 서비스의 큰 그림
- 2-2. 한 문장 요약
- 2-3. 각 서비스의 "담당 업무"

3장. 기술 스택 지도

- 3-1. 역할별 총정리
- 3-2. 왜 이 조합인가? (대안 비교)

4장. WALL 1 — 개발 환경 세팅

- 4-1. IDE (코드 에디터) — Antigravity
- 4-2. Node.js — JavaScript 실행기
- 4-3. npm — 레고 부품 구매 시스템
- 4-4. Next.js + TypeScript + Tailwind CSS
- 4-5. Git / GitHub — 코드 타임머신
- 4-6. GitHub 연동 3단계
- 4-7. 실전 주의사항 — Vercel 이메일 매칭 에러
- 4-8. UI 레퍼런스 찾는 법
- ✓ WALL 1 체크리스트

5장. WALL 2 — 인증 (로그인)

- 5-1. 왜 로그인이 필요한가?
- 5-2. Supabase — 백엔드 통째로 제공하는 서비스
- 5-3. Supabase의 "키" 2가지 — 절대 헛갈리면 안 됨
- 5-4. OAuth — 비밀번호 없이 로그인하는 방법
- 5-5. Supabase + Google OAuth 연동 흐름 (3자 관계)
- 5-6. JWT — 로그인 증명서
- 5-7. AuthContext — 앱 전체에서 로그인 상태 공유
- 5-8. 배포 후 필수 설정
- 5-9. Supabase MCP — AI 에이전트의 DB 직접 접근
- ✓ WALL 2 체크리스트

6장. 핵심 기능 — AI 채팅

- 6-1. Gemini API — 구글의 AI 두뇌 빌려쓰기
 - 6-2. 왜 API Route(서버)에서 호출하는가?
 - 6-3. 스트리밍 응답 — 타자 치듯 나오는 답변
 - 6-4. 대시보드 UI 구조
 - 6-5. 대화 저장 (Supabase DB)
 - 6-6. RLS (Row Level Security) — 반드시 이해해야 하는 보안 개념
 - 6-7. 채팅 플로우 (시퀀스 다이어그램)
- ✓ 핵심 기능 체크리스트

7장. WALL 3 — 결제

- 7-1. Polar — SaaS 결제 플랫폼
 - 7-2. Sandbox vs Production
 - 7-3. Polar 셋업 순서
 - 7-4. 결제 흐름 전체 그림
 - 7-5. Vercel 배포 (결제 테스트 전에 해야!)
 - 7-6. Webhook — 결제 이벤트 알림
 - 7-7. Customer Portal — 구독 관리 페이지
- ✓ WALL 3 체크리스트

8장. 구독 설계 — 플랜과 사용량 관리

- 8-1. 플랜 구성
 - 8-2. 구독 라이프사이클 (전체 흐름)
 - 8-3. Webhook 이벤트 종류 — canceled vs revoked 구분 필수
 - 8-4. DB 스키마 — subscriptions 테이블
 - 8-5. 사용량 추적 로직
 - 8-6. /api/chat에서 제한 적용
 - 8-7. UX — 사용량 표시
- ✓ 구독 설계 체크리스트

9장. 테스트

- 9-1. 테스트는 배포된 도메인에서
- 9-2. MVP 테스트 매트릭스

10장. 암호화 — 유저 정보 보호

- 10-1. 왜 암호화?
 - 10-2. AES-256-GCM — 암호화 알고리즘
 - 10-3. 해시 vs 암호화 — 헷갈리지 마세요
 - 10-4. 키 2개 — ENCRYPTION_KEY, HASH_KEY
 - 10-5. 절대 규칙 — 키 관리
 - 10-6. 구현 요소 5가지
 - 10-7. 검증 방법
- ✓ 암호화 체크리스트

11장. 환경변수 총정리

- 11-1. 전체 목록
- 11-2. 접두사 규칙 요약
- 11-3. 2곳 동기화 — 가장 흔한 실수

12장. AI 프롬프트 작성법

- 12-1. 좋은 프롬프트 공식
- 12-2. 좋은 예시 vs 나쁜 예시
- 12-3. 디버깅 프롬프트 공식
- 12-4. UI 프롬프트 공식

13장. 자주 겪는 에러 사전

14장. 용어 사전

- A. 기본 웹 개발 용어
- B. 개발 일반
- C. API & 통신
- D. Next.js & React
- E. 인증 & 보안
- F. 데이터베이스
- G. 결제 & 구독
- H. AI & LLM
- I. 암호학
- J. 운영 & 배포

15장. 다음 단계 학습 로드맵

- 15-1. 깊이 학습 (이미 쓴 기술 더 파보기)
- 15-2. 넓이 확장 (인접 기술)
- 15-3. 비즈니스 관점

16장. 최종 정리

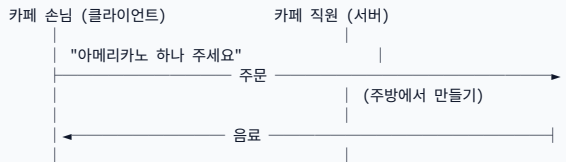
- 머릿속에 남겨야 할 3가지
- 벽은 생각보다 낮습니다
- 다음 프로젝트에선...

0장. 코딩 용어 맛보기

1장 들어가기 전에, 문서 전체에서 반복해서 나올 핵심 단어를 먼저 정리합니다.

0-1. 클라이언트 vs 서버 — 가장 중요한 구분

일상 비유: 카페에서 주문하기



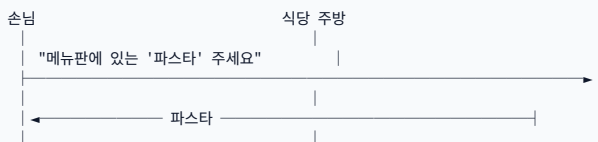
클라이언트	서버
웹 서비스를 사용하는 쪽 (손님)	웹 서비스를 제공하는 쪽 (직원)
보통 브라우저 에서 실행	보통 회사 컴퓨터 에서 실행
예: 크롬, 사파리	예: Vercel, AWS, Supabase

왜 이 구분이 중요한가?

- 시크릿 키(비밀번호)는 서버에만 있어야 함 → 클라이언트에 넣으면 누구나 훔쳐볼 수 있음
- 데이터베이스 접근도 서버에서만 → 클라이언트가 직접 DB에 접근하면 위험
- 이 구분을 모르면 보안 사고가 납니다

0-2. API — 프로그램끼리 대화하는 방법

일상 비유: 식당 메뉴판



- **메뉴판** = API 명세서: "이런 걸 주문할 수 있고, 이런 게 나와요"
- **주문** = API 요청(Request): 정해진 형식으로 요청
- **음식** = API 응답(Response): 정해진 형식으로 돌아옴

우리 프로젝트의 API 예시

주문 (요청)	음식 (응답)
POST /api/chat + 메시지 내용	Gemini가 생성한 AI 답변
POST /api/checkout + 플랜 이름	Polar 결제 페이지 URL

0-3. URL, 도메인, 라우트

https://your-app.vercel.app/pricing?plan=pro

프로토콜 도메인(어디인지) 경로 쿼리 (추가 정보)

- **도메인:** 웹사이트 주소 (예: `google.com`, `your-app.vercel.app`)
- **라우트(경로):** 도메인 뒤에 붙는 부분 (`/pricing`, `/dashboard`)
- Next.js에서는 **폴더 이름이 곧 라우트:** `app/pricing/page.tsx` → `/pricing` 페이지

0-4. 환경변수 — 비밀번호 보관함

일상 비유: 현관문 비밀번호

- 친구에게 집 주소는 알려줘도 되지만 (`NEXT_PUBLIC_...`),
- 현관 비밀번호는 비밀 (`SUPABASE_SECRET_KEY`)
- 이 비밀번호들을 코드 안에 직접 쓰면 → 코드 공유할 때 유출됨
- 그래서 **별도 파일** (`.env.local`)에 넣어두고, 코드에서는 "환경변수로 가져와"

코드
↓ `process.env.POLAR_ACCESS_TOKEN`
.env.local 파일
`POLAR_ACCESS_TOKEN=polar_oat_xxxxx` ← 여기만 실제 값

0-5. 데이터베이스(DB) — 정보를 저장하는 엑셀

비유: 거대한 엑셀 파일

conversations 테이블 (대화 목록)

id	user_id	title	created_at
1	A	"안녕"	2026-04-20
2	A	"파이썬 질문"	2026-04-21
3	B	"여행 계획"	2026-04-21

- **테이블:** 엑셀 시트 하나 (예: `conversations`, `messages`)
- **컬럼(열):** 데이터 종류 (id, 제목, 생성 시간...)
- **로우(행):** 실제 데이터 한 개

0-6. 프론트엔드 vs 백엔드

프론트엔드 (Front-end)	백엔드 (Back-end)
사용자가 보는 화면	사용자가 안 보는 데이터 처리
버튼, 텍스트, 색깔, 애니메이션	DB 저장, AI 호출, 결제 처리
= 클라이언트에서 대부분 실행	= 서버에서 실행

Next.js는 둘 다 한 번에 할 수 있는 프레임워크 → 그래서 유용함

0-7. 이제 본문으로

이 7가지 개념만 익혔으면 충분합니다. 나머지는 본문에서 **하나씩 비유와 함께** 설명합니다. 모르는 단어가 나오면 14장 **용어 사전**에서 찾을 수 있습니다.

1장. 바이브 코딩이란?

1-1. 핵심 철학

"벽은 생각보다 낮다"

SaaS 서비스를 만드는 3개의 벽 — 시작(개발 환경), 인증, 결제 — 만 넘으면 누구나 런칭 가능.

전통적 개발 vs 바이브 코딩

구분	전통적 개발	바이브 코딩
코드 작성	모든 줄을 내가 직접	AI에게 자연어로 "이렇게 만들어줘"
필요한 지식	언어·프레임워크 문법 외우기	아키텍처 이해 + 서비스 연결
핵심 역량	코드 작성 속도	문제 정의·검수 능력
비유	요리사가 직접 조리	요리사에게 "이런 요리" 주문 + 검수

1-2. 바이브 코딩의 3원칙

① 완벽보다 일단 돌아가게

- ✕ 처음부터 UI 완벽히 다듬기
- ✓ 일단 기능이 작동하게 만들고 → 나중에 예쁘게

이유: 작동 안 하는 예쁜 서비스는 쓸모없음. 작동하는 못생긴 서비스는 고칠 수 있음.

② 에러는 그대로 복붙해서 AI에 전달

- ✕ 구글 검색 → 스택오버플로우 찾기 → 30분 소요
- ✓ 에러 메시지 그대로 복사 → AI에 붙여넣기 → 30초 해결

③ 모르면 작게 쪼개서 물어보기

- ✕ "SaaS 서비스 전체를 만들어줘"
- ✓ "로그인 페이지만 먼저 만들어줘"
- 성공 확인 → "대시보드 만들어줘" → ...

1-3. 내가 할 일 vs AI가 할 일

▲ 사람 (나)	
• 무엇을 만들지 결정 • 전체 구조 설계 • 외부 서비스 계정 생성·키 발급 • 검수·테스트·배포 결정	
↕	
● AI 에이전트	
• 코드 작성 • 라이브러리 사용법 조사 • 에러 해석 및 해결 제안 • UI 스타일링·리팩토링	

2장. 전체 시스템 한눈에 보기

2-1. 완성된 서비스의 큰 그림



2-2. 한 문장 요약

사용자가 **Next.js 앱**에 접속 → **Supabase**로 구글 로그인 → **Gemini**로 AI 채팅 → **Polar**로 결제 → 모든 데이터는 **Supabase DB**에 저장 → 전체는 **Vercel**에 배포.

2-3. 각 서비스의 "담당 업무"

Vercel	= "우리 앱을 인터넷에 올려주는 집"
Next.js	= "집 안의 인테리어와 시스템"
Supabase	= "고객 명부 + 대화 기록 저장소 + 로그인 창구"
Google	= "OAuth 신분증 발급 기관"
Gemini	= "AI 답변 해주는 똑똑한 직원"
Polar	= "결제 담당 직원 + 계산서 관리"
GitHub	= "모든 작업 히스토리 저장소 (타임머신)"

3장. 기술 스택 지도

3-1. 역할별 총정리

역할	기술	무료 시작	한 줄 설명
IDE (에디터)	Antigravity / Cursor / Claude Code	✓	AI 내장된 VS Code
언어	TypeScript	✓	JavaScript + 타입 체크
프레임워크	Next.js	✓	React 기반 풀스택
스타일	Tailwind CSS	✓	유틸리티 CSS
런타임	Node.js	✓	서버에서 JS 실행
버전관리	Git + GitHub	✓	코드 타임머신
인증 + DB	Supabase	✓ (2GB)	백엔드 통째로
AI	Gemini API	✓ 넉넉함	Google의 AI
결제	Polar	✓ Sandbox	SaaS 결제
배포	Vercel	✓	Next.js 자동 배포
UI 참고	21st.dev	✓	컴포넌트 갤러리

3-2. 왜 이 조합인가? (대안 비교)

선택	대안	이 조합의 장점
Next.js	Vue, Angular	Vercel 배포가 원클릭, 생태계 크고 AI가 잘 압
Supabase	Firebase	표준 PostgreSQL → 나중에 다른 DB로 이전 가능
Gemini	OpenAI GPT	무료 티어 넉넉, 한국에서도 안정적
Polar	Stripe	가입 쉽고, SaaS 구독에 특화
Vercel	AWS	Next.js 제작사가 만든 플랫폼, 무설정

4장. WALL 1 — 개발 환경 세팅

3가지만 짚면 끝: Antigravity + Node.js + GitHub

4-1. IDE (코드 에디터) — Antigravity

뭐냐? AI 에이전트가 내장된 코드 에디터.

비유: 마이크로소프트 워드(텍스트 쓰는 프로그램) + AI 비서(자동으로 글 써주는)가 합쳐진 것.

항목	설명
기반	VS Code (세계에서 가장 많이 쓰는 에디터)를 포크한 것
차이점	Gemini AI 에이전트 내장 (무료)
대안	Cursor, Claude Code, Windsurf

에이전트 정책 2가지

- **Review-driven (권장)**: AI가 "이렇게 바꿀게요" 제안 → 내가 OK 누르면 실행
- **Auto-apply**: AI가 알아서 수정 → 빠르지만 실수 누적 위험

Agent Manager 패널: 왼쪽에 있는 채팅창. 여기서 AI에게 지시하면 코드를 바로 작성·수정함.

4-2. Node.js — JavaScript 실행기

뭐냐? JavaScript 코드를 브라우저가 아닌 곳에서 실행하게 해주는 프로그램.

비유:

- JavaScript = 영어로 쓴 편지
- 브라우저 = 영어를 읽을 수 있는 친구
- Node.js = 영어 번역기 → 이제 다른 친구(터미널, 서버)도 편지를 읽을 수 있음

왜 설치해야?

- `npm` 명령어를 쓰려면 Node.js가 필요 (npm은 Node.js에 포함)
- Next.js 프로젝트 생성·실행도 Node.js 위에서 돌아감

확인 방법

```
node -v    # v20.11.0 같은 결과가 나오면 OK
npm -v     # 10.2.4 같은 결과가 나오면 OK
```

4-3. npm — 레고 부품 구매 시스템

뭐냐? Node.js의 패키지 매니저 (라이브러리 설치 도구).

비유: 레고 부품 상점.

- 내가 우주선을 만들려고 할 때, 부품을 하나하나 깎는 게 아니라 상점에서 구매
- `npm install @polar-sh/sdk` = "Polar 결제 부품 하나 주세요"

주요 파일

파일	역할	비유
<code>package.json</code>	필요한 부품 목록	레고 설명서의 "필요한 부품" 페이지
<code>package-lock.json</code>	정확한 부품 버전	부품 번호까지 명시된 주문서
<code>node_modules/</code>	실제 다운로드된 부품 보관함	레고 박스 안

흔한 명령어

```

npm install          # package.json에 적힌 부품 전부 설치
npm install <pkg>    # 새 부품 추가
npm run dev          # 개발 서버 실행 (localhost:3000)
npm run build        # 배포용으로 빌드

```

주의: `node_modules/` 는 용량이 어마어마합니다 (수백 MB). `.gitignore` 에 있어서 깃허브에 안 올라가고, 필요하면 `npm install` 로 재설치하면 됨.

4-4. Next.js + TypeScript + Tailwind CSS

Next.js — 집짓는 틀(프레임워크)

비유: 조립식 주택 키트.

- 맨땅부터 집 짓는 게 아니라, 기본 골격이 이미 있는 키트로 시작
- 우리는 인테리어만 꾸미면 됨

Next.js의 강점

1. 파일 이름이 곧 URL (App Router 방식)

`app/page.tsx` → 홈페이지 (/) `app/pricing/page.tsx` → `/pricing` 페이지 `app/api/chat/route.ts` → `/api/chat` API

2. 프론트엔드 + 백엔드를 한 프로젝트에서: 따로 서버를 만들 필요 없음
3. Vercel과 궁합 최적: 배포가 원클릭

TypeScript — 오타 방지 맞춤법 검사기

JavaScript vs TypeScript

```

// JavaScript (타입 없음)
const age = "25"          // 문자열 "25"
age + 1                   // "251" (버그!)

// TypeScript (타입 있음)
const age: number = "25"  // ✕ 컴파일 에러: "숫자여야 하는데 문자열"

```

왜 중요한가?

- AI가 코드를 짜다가 타입 실수를 하면 TypeScript가 잡아줌
- `npx tsc --noEmit` 명령으로 전체 프로젝트 타입 검사 가능

Tailwind CSS — 유틸리티 옷장

전통 CSS vs Tailwind

```
<!-- 전통 CSS (별도 파일) -->
<button class="my-button">클릭</button>
<style>
  .my-button {
    padding: 16px;
    background: black;
    color: white;
    border-radius: 8px;
  }
</style>

<!-- Tailwind (HTML에 직접) -->
<button class="p-4 bg-black text-white rounded-lg">클릭</button>
```

왜 Tailwind?

- CSS 파일을 따로 만들 필요 없음 → AI가 HTML만 수정해도 스타일 완성

- 다크 테마 쉬움: `bg-white dark:bg-neutral-950`

4-5. Git / GitHub — 코드 타임머신

비유: 게임의 세이브 포인트 + 드롭박스.

오늘 3시: 커밋 "로그인 추가" ← 세이브 1
오늘 4시: 커밋 "채팅 추가" ← 세이브 2
오늘 5시: ■ AI가 코드 망침
오늘 5시: 세이브 2로 롤백 → 복구!

핵심 용어

용어	의미	비유
레포(Repository)	프로젝트의 버전 기록 저장소	게임 저장 파일 폴더
커밋(Commit)	특정 시점의 스냅샷 + 설명	세이브 포인트
브랜치(Branch)	평행 작업선	"만약에 이렇게 했다면" 평행 우주
main	기본 브랜치 (공식 배포용)	메인 세계관
push	로컬 → GitHub 업로드	드롭박스 동기화
pull	GitHub → 로컬 다운로드	드롭박스에서 가져오기

절대 커밋 금지 파일

`.env.local` ← API 키 들어있음. 유출되면 해커 파티
`node_modules/` ← 용량 어마어마. 재설치 가능

→ `.gitignore` 파일에 기재해서 Git이 무시하게 함

커밋 메시지 컨벤션 (Conventional Commits)

접두사	의미	예시
<code>feat:</code>	새 기능	<code>feat: add pricing page</code>
<code>fix:</code>	버그 수정	<code>fix: resolve login redirect loop</code>
<code>chore:</code>	설정·빌드	<code>chore: update dependencies</code>
<code>refactor:</code>	기능 그대로, 코드 정리	<code>refactor: extract auth helpers</code>
<code>docs:</code>	문서만	<code>docs: update README</code>

4-6. GitHub 연동 3단계

1. GitHub.com에서 새 repository 생성
(public / private, name: my-gemini-wrapper)
2. 로컬 터미널에서:

```
git remote add origin https://github.com/[내아이디]/my-gemini-wrapper.git
git branch -M main
git push -u origin main
```
3. GitHub 페이지 새로고침 → 내 코드 보이면 성공!

4-7. 실전 주의사항 — Vercel 이메일 매칭 에러

증상: 배포할 때 `commit email could not be matched to a GitHub account` 에러

원인: Git 커밋에 찍힌 이메일이 내 GitHub 계정에 등록되었지 않음

확인

```
git config --get user.email
```

해결 방법 3가지

1. GitHub 설정 → Emails에 그 이메일 추가 (추천)
2. Git 이메일을 GitHub 등록 이메일로 변경:

```
bash git config --global user.email "[GitHub에 등록된 이메일]"
```

3. Vercel 프로젝트 → Settings → Git → "Author Verification" 끄기

4-8. UI 레퍼런스 찾는 법

21st.dev — React 컴포넌트 갤러리

- 고품질 UI 샘플 → 맘에 드는 걸 찾아서 AI에게 "이렇게 만들어줘"

좋은 UI 프롬프트 공식

```
[레퍼런스 스크린샷/링크]
+ [만들 목적: "Gemini 래퍼 서비스의 히어로 섹션"]
+ [제약: "다크 테마, 심플하게"]
+ [세부: "타이틀은 X, CTA 버튼 2개 (Get started, Pricing)"]
```

✓ WALL 1 체크리스트

- ☐ Antigravity 설치 + Review-driven 모드 선택
- ☐ Node.js LTS 설치 (`node -v`로 확인)
- ☐ Next.js 프로젝트 생성 (TS + Tailwind + App Router)

- ☐ GitHub 계정 생성 + 새 레포 생성
 - ☐ `git remote add` → `push` 연동
 - ☐ `.env.local` 이 `.gitignore` 에 포함됐는지 확인
 - ☐ 랜딩 페이지 히어로 섹션 만들기
-

5장. WALL 2 — 인증 (로그인)

사용자가 누구인지 증명하는 단계. **Supabase + Google OAuth**로 해결.

5-1. 왜 로그인이 필요한가?

로그인 없는 서비스:

- ✕ 누가 결제했는지 모름
- ✕ 개인 대화 기록 저장 불가
- ✕ 한 사람이 무제한 사용 가능

로그인 있는 서비스:

- ✓ 유저별 데이터 분리
- ✓ 구독 플랜 관리 가능
- ✓ 사용량 제한 가능

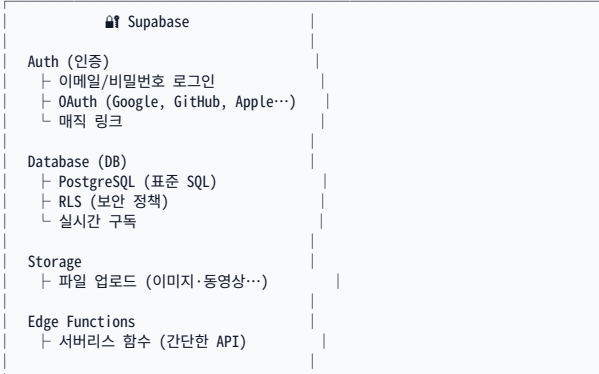
5-2. Supabase — 백엔드 통째로 제공하는 서비스

뭐냐? 백엔드 서비스 묶음. (백엔드 = 사용자 눈에 안 보이는 서버 부분)

비유: 편의점에서 도시락 사기.

- 직접 요리할 수도 있지만 (백엔드 서버 직접 구축)
- 편의점 도시락 사면 완성된 밥+반찬+국 다 있음 (Supabase)

Supabase가 주는 것들



우리 프로젝트는 Auth + DB만 사용

5-3. Supabase의 "키" 2가지 — 절대 헛갈리면 안 됨

키 이름	사용 위치	노출 시 위험
Publishable Key (이름에 <code>NEXT_PUBLIC_</code>)	클라이언트(브라우저) OK	낮음 — RLS가 막아줌
Secret Key / Service Role Key	서버 전용	치명적 — DB 전체 탈취 가능

비유:

- Publishable = 집 주소 (공개 OK)
- Secret = 금고 마스터 키 (절대 공개 금지)

⚠ 시크릿 키에 `NEXT_PUBLIC_` 접두사를 붙이면 클라이언트에 그대로 노출됩니다. 절대 붙이지 마세요.

5-4. OAuth — 비밀번호 없이 로그인하는 방법

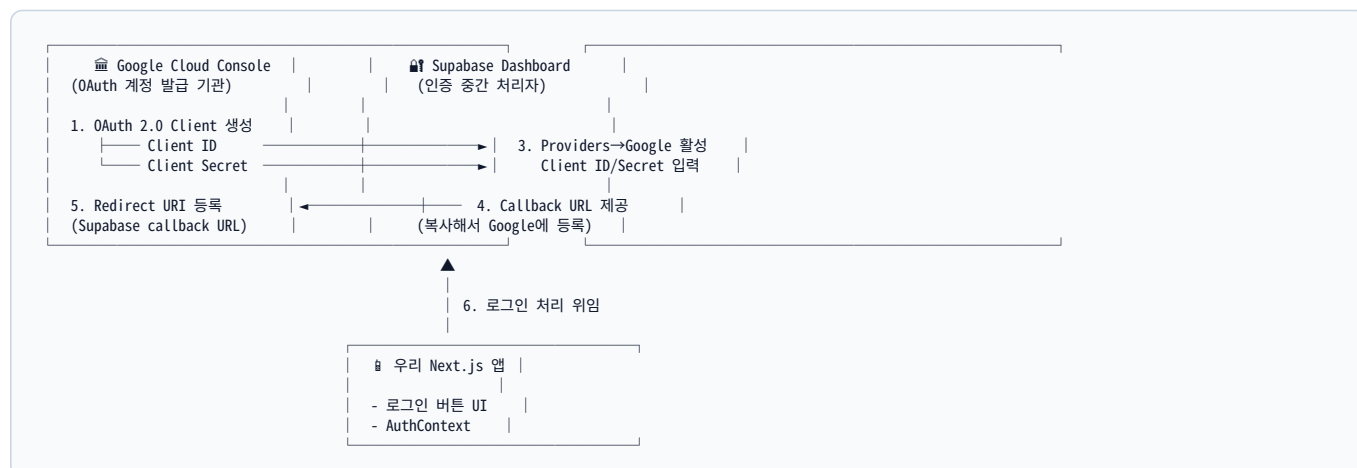
뭐냐? "우리 앱에 구글 비밀번호를 직접 주지 않고도 구글 계정으로 로그인"하게 해주는 표준.

비유: 나이트클럽 입장



- 우리 앱은 **비밀번호를 모름** → 안전
- 구글이 "이 사람 진짜예요"라고 보증

5-5. Supabase + Google OAuth 연동 흐름 (3자 관계)



실전 순서

1. Supabase 프로젝트 생성 → URL, anon key 복사
2. Google Cloud Console에서 OAuth 클라이언트 생성
3. Google에서 Client ID + Client Secret 받기
4. Supabase → Auth → Providers → Google 활성화 → ID/Secret 붙여넣기
5. Supabase가 알려주는 Redirect URL을 Google Console에 다시 등록
6. 우리 앱 코드에서 `supabase.auth.signInWithOAuth({ provider: "google" })` 호출

5-6. JWT — 로그인 증명서

뭐냐? 로그인 성공 시 Supabase가 발급하는 암호화된 증명서.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjMONTY3ODkwIi... 헤더 . pay load(내용) . 서명
```

비유: 영화관 티켓.

- 티켓에는 좌석번호(payload)가 찍혀있지만 (읽을 수 있음)
- 위조하면 OR 스캐너가 알아봄 (서명이 다름)

JWT의 세 부분

부분	내용	비유
Header	암호화 알고리즘 정보	티켓 용지 종류
Payload	user_id, email, 만료시간	좌석 번호
Signature	Supabase 비밀번호로 서명	위조 방지 홀로그램

우리 코드에서 JWT 사용

```
// 클라이언트: JWT 꺼내서 API에 첨부
const { data } = await supabase.auth.getSession();
const token = data.session?.access_token;

await fetch("/api/chat", {
  headers: { Authorization: `Bearer ${token}` }
});

// 서버: JWT 검증
const { data: userData } = await supabase.auth.getUser(token);
// → userData.user.id, userData.user.email 획득
```

5-7. AuthContext — 앱 전체에서 로그인 상태 공유

문제: 로그인 상태를 어떻게 모든 페이지에서 알 수 있나?

해결: React Context 사용 (앱 전체의 "공용 메모장")

```
app/layout.tsx
├── <AuthProvider> ← 여기서 세션 관리
│   ├── 페이지 1 → useAuth() 호출 → { user, loading, signOut }
│   ├── 페이지 2 → useAuth() 호출 → 같은 정보 공유
│   └── 페이지 3 → useAuth() 호출 → 같은 정보 공유
```

사용 예

```
const { user, loading, signInWithGoogle, signOut } = useAuth();

if (loading) return <Loading />;
if (!user) return <LoginPrompt />;
return <Dashboard user={user} />;
```

5-8. 배포 후 필수 설정

로컬에서 잘 되던 로그인이 배포 후 안 되면 99% 이 두 가지:

1. Supabase → Authentication → URL Configuration

- Site URL: `https://your-app.vercel.app`
- Redirect URLs: `https://your-app.vercel.app/**` (와일드카드 권장)

2. Google Cloud Console → 승인된 Redirect URI

- Supabase가 알려주는 callback URL + 우리 배포 도메인 둘 다 등록

5-9. Supabase MCP — AI 에이전트의 DB 직접 접근

MCP란? Model Context Protocol. AI 에이전트가 **Supabase**에 **직접 명령**을 내릴 수 있게 하는 연결 규격.

MCP 없을 때 vs MCP 있을 때

✕ MCP 없음:
 나: "conversations 테이블 만들어줘"
 AI: "이 SQL을 Supabase 대시보드에 복사해서 실행하세요: CREATE TABLE..."
 나: (복사 → 대시보드 → 붙여넣기 → 실행)

✓ MCP 있음:
 나: "conversations 테이블 만들어줘"
 AI: (자동으로 Supabase에 실행) "완료했습니다!"

설정 (`.mcp.json` 파일)

```
{
  "mcpServers": {
    "supabase": {
      "url": "https://mcp.supabase.com/mcp"
    }
  }
}
```

MCP 제공 기능

기능	설명
테이블 관리	생성·조회·마이그레이션
SQL 실행	에이전트가 직접 쿼리
로그 확인	에러 원인 분석
TypeScript 타입 자동 생성	DB 스키마 기반

안전 옵션

- `read_only=true` : 읽기만 (실수 방지)
- `project_ref=[ID]` : 특정 프로젝트만
- 프로덕션 DB에는 절대 직접 연결 금지

✓ WALL 2 체크리스트

- ☐ Supabase 프로젝트 생성 → URL, anon key 복사
- ☐ Google Cloud Console에서 OAuth 클라이언트 생성
- ☐ Client ID/Secret 복사 → Supabase Providers에 입력
- ☐ Supabase callback URL → Google 승인된 URI에 등록
- ☐ `.env.local` 에 환경변수 저장
- ☐ AuthProvider + useAuth 혹 구현
- ☐ 로그인 페이지 + 구글 버튼
- ☐ 미로그인 시 `/dashboard` → `/login` 리다이렉트
- ☐ 로그아웃 기능
- ☐ (선택) Supabase MCP 연결

스트리밍 응답

User: "한국사 요약해줘"
 AI: "한" → "한국" → "한국의" → "한국의 역사는..."
 (실시간 글자 단위로 도착)

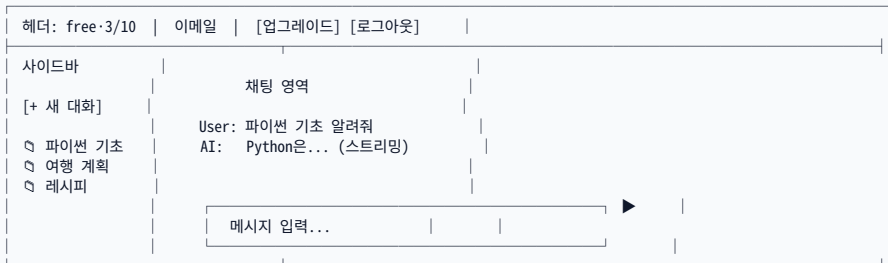
왜 중요?

- 사용자 체감 속도 ↑ (실제 속도는 같아도 "답변 중"이 보여서 안 지루함)
- ChatGPT 같은 느낌 → 자연스러운 UX

구현 원리

서버:
 ReadableStream 생성 → AI 답변 토큰별로 controller.enqueue(...)
 클라이언트:
 fetch 응답의 body.getReader()로 조각별로 읽기 → UI 업데이트

6-4. 대시보드 UI 구조



6-5. 대화 저장 (Supabase DB)

테이블 설계

conversations (대화 목록)

id	user_id	title	created_at
uuid-aaa	uuid-user1	"파이썬 기초"	2026-04-20
uuid-bbb	uuid-user1	"여행 계획"	2026-04-21
uuid-ccc	uuid-user2	"레시피"	2026-04-21

messages (메시지)

id	conversation_id	role	content	created_at
uuid-1	uuid-aaa	user	"파이썬이 뭐?"	2026-04-20
uuid-2	uuid-aaa	assistant	"Python은..."	2026-04-20
uuid-3	uuid-bbb	user	"제주도 가려고"	2026-04-21

핵심 용어

용어	의미	비유
PK (Primary Key)	고유 식별 번호	주민등록번호
FK (Foreign Key)	다른 테이블 참조	"엄마가 누구?" 참조
uuid	전세계 유일한 ID (임의 문자열)	자동 주민번호 발급
ON DELETE CASCADE	부모 삭제 → 자식 자동 삭제	대화 삭제 → 메시지 전부 자동 삭제
INDEX	검색 속도 향상 구조	책 뒷편 색인

6-6. RLS (Row Level Security) — 반드시 이해해야 하는 보안 개념

RLS 없을 때의 위험

User A의 브라우저에서:
 SELECT * FROM conversations WHERE user_id = 'user_B_id'
 → User B의 모든 대화 조회 가능 △ 심각한 사고!

RLS 있을 때

User A의 브라우저에서:
 SELECT * FROM conversations WHERE user_id = 'user_B_id'
 → DB가 자동 필터: "A는 자기 것만 봐야지" → 0행 반환 ✓

RLS 정책 예시

```
alter table public.conversations enable row level security;

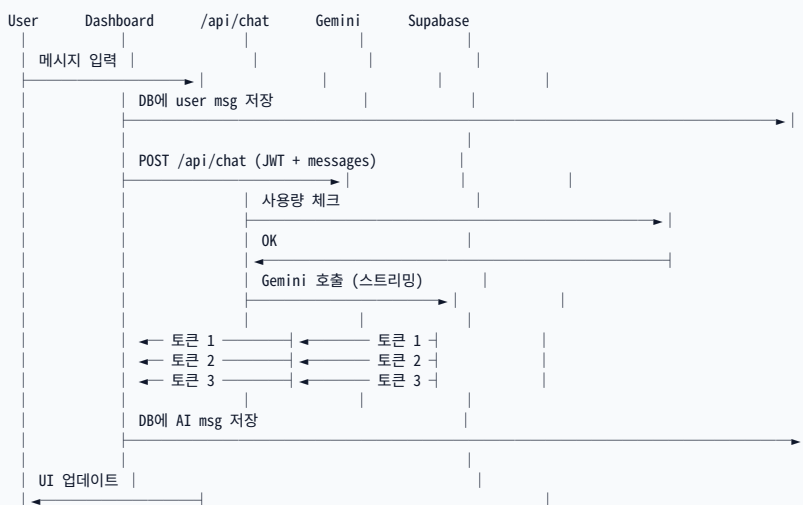
create policy "conversations_owner" on public.conversations
for all
using (auth.uid() = user_id)      -- 읽기·수정·삭제 조건
with check (auth.uid() = user_id); -- 쓰기 조건
```

용어	의미
<code>auth.uid()</code>	현재 요청자의 user_id (JWT에서 추출)
<code>using</code>	SELECT/UPDATE/DELETE에 적용
<code>with check</code>	INSERT/UPDATE 시 "넣을 수 있는 값" 조건

자주 보는 에러

Error: "new row violates row-level security policy"
 → 원인: 정책이 없거나, auth.uid()와 user_id가 안 맞음
 → 해결: 에이전트에게 "RLS 정책 SQL 만들어줘" 요청

6-7. 채팅 플로우 (시퀀스 다이어그램)



✓ 핵심 기능 체크리스트

- ☐ Gemini API 키 발급 → `.env.local`
 - ☐ `/api/chat` Route Handler 구현 (스트리밍)
 - ☐ 대시보드 레이아웃 (사이드바 + 채팅)
 - ☐ `conversations`, `messages` 테이블 생성
 - ☐ 각 테이블 RLS 정책 활성화
 - ☐ 대화 생성·조회·삭제
 - ☐ 메시지 저장 + 히스토리 로드
 - ☐ 스트리밍 응답 UI 확인
-

7장. WALL 3 — 결제

돈 받는 단계. **Polar**로 결제. **배포 먼저 → 결제 테스트** 순서.

7-1. Polar — SaaS 결제 플랫폼

뭐냐? 월 구독형 소프트웨어(SaaS) 결제에 특화된 서비스.

비유: 넷플릭스의 결제 시스템을 대여해서 쓰는 것.

항목	Polar	Stripe (대안)
가입	GitHub 계정 OK	사업자 정보 필요
UI	단순	복잡하지만 강력
구독 관리	쉬움	유연하지만 학습곡선 ↑
한국 결제	가능	가능

7-2. Sandbox vs Production

환경	의미	사용처
Sandbox	가짜 환경 (테스트용)	개발 · 테스트
Production	실제 환경 (진짜 돈)	배포 후 실사용

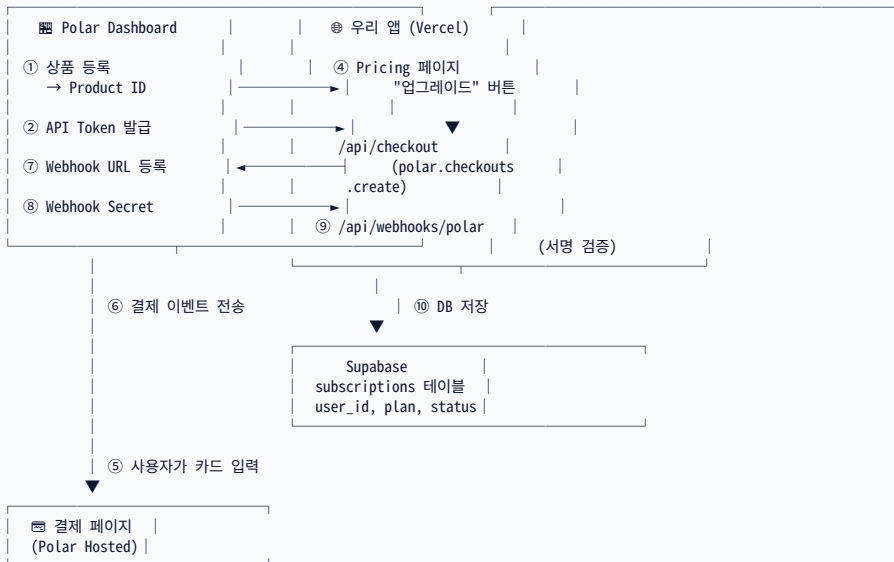
우리 프로젝트는 **Sandbox**로 진행. 진짜 카드번호를 넣어도 돈이 안 나감.

7-3. Polar 셋업 순서

1. polar.sh 가입 (GitHub 로그인)
2. 왼쪽 하단 Sandbox로 전환
3. 상품 등록
 - Pro (\$9.99/월) → Product ID 복사
 - Unlimited (\$29.99/월) → Product ID 복사
4. Settings → API Tokens → Polar API Token 발급
(polar_oat_xxxxx 형식)
5. .env.local에 저장:


```
POLAR_ACCESS_TOKEN=polar_oat_...
POLAR_PRODUCT_ID_PRO=...
POLAR_PRODUCT_ID_UNLIMITED=...
POLAR_SERVER=sandbox
```

7-4. 결제 흐름 전체 그림



7-5. Vercel 배포 (결제 테스트 전에 해야!)

왜 배포 먼저?

- Webhook을 받으려면 **서버가 인터넷에 공개**돼야 함
- 로컬(`localhost`)은 외부에서 접근 불가
- 배포하면 `https://your-app.vercel.app` 같은 공개 URL 생김

Vercel이란?

- Next.js 제작사가 만든 배포 플랫폼
- `git push` 하면 자동으로 빌드 + 배포
- 무료 플랜으로 개인 프로젝트 충분

배포 순서

1. `git push` (GitHub에 올림)
2. `vercel.com` → New Project → GitHub repo Import
3. 환경변수 전부 입력 (로컬 `.env.local` 내용 복사)
4. Deploy 클릭
5. 완료되면 URL 확인 (예: `my-app.vercel.app`)

배포 후 필수 작업

- ☐ Supabase → Auth → URL Configuration에 배포 도메인 추가
- ☐ Google Cloud Console 승인된 Redirect URI에 배포 도메인 추가

환경변수 바꿨을 때

- Vercel은 자동 재배포 안 함
- Settings → Environment Variables 수정 후 **Redeploy** 필요

7-6. Webhook — 결제 이벤트 알림

뭐냐? 외부 서비스가 "이벤트 발생!"을 우리 서버로 알려주는 HTTP 요청.

비유: 택배 도착 알림.

- 택배 회사(Polar)가 "결제 완료 택배 도착!"을 우리 집(서버) 주소로 보냄
- 우리 집 문(`/api/webhooks/polar`)에서 받아서 처리

왜 필요한가?

- ✕ 웹훅 없음:
클라이언트: "저 결제 완료했어요" → 서버: "정말이야? 믿을 수 있나..."
→ 조작 가능 (악의적 유저가 거짓말 가능)
- ✓ 웹훅 있음:
Polar 서버: "[서명] 이 유저 결제 완료됨" → 우리 서버: "서명 맞네, 진짜구나"
→ 위조 불가

웹훅 서명 검증

```
import { validateEvent } from "@polar-sh/sdk/webhooks";

try {
  const event = validateEvent(rawBody, headers, POLAR_WEBHOOK_SECRET);
  // 서명 OK → 처리 계속
} catch {
  return new Response("invalid signature", { status: 403 });
}
```

웹훅 등록 순서

1. Polar Dashboard → Webhooks → 엔드포인트 추가
2. URL: `https://your-app.vercel.app/api/webhooks/polar`
3. 구독할 이벤트 선택 (`subscription.*`)
4. 생성된 **Webhook Secret** 복사
5. `.env.local` + Vercel 환경변수에 `POLAR_WEBHOOK_SECRET` 저장
6. Vercel 재배포

7-7. Customer Portal — 구독 관리 페이지

뭐냐? 사용자가 직접 구독 관리하는 페이지 (결제 수단 변경, 취소, 청구서).

비유: 넷플릭스의 "내 계정" 페이지. Polar가 자체 제공.

구현

- Billing Settings 페이지에 "Polar Customer Portal" 링크 버튼
- Polar SDK의 `customerPortal` 기능으로 링크 생성
- 현재 구독 상태, 다음 결제일 표시

✓ WALL 3 체크리스트

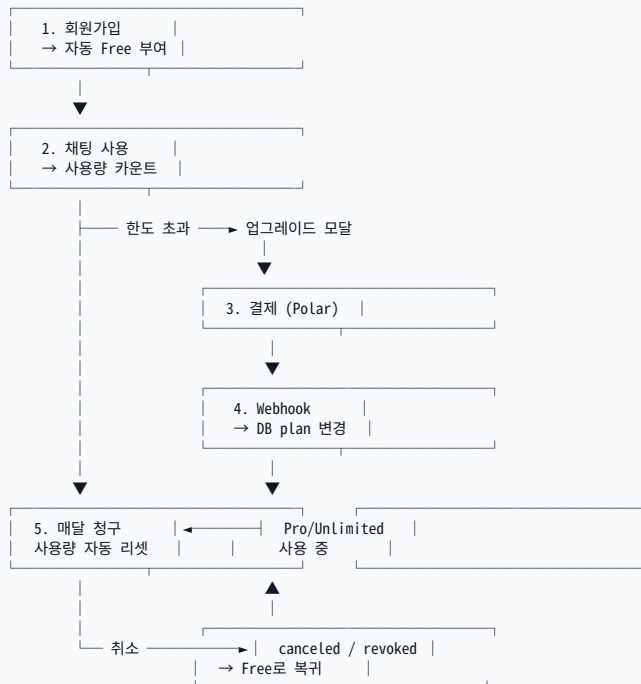
- ☐ Polar 가입 + Sandbox 전환
- ☐ Pro / Unlimited 상품 등록
- ☐ Product ID + API Token 발급
- ☐ `.env.local` 에 Polar 환경변수 저장
- ☐ Pricing 페이지 UI + 업그레이드 버튼
- ☐ `/api/checkout` API 구현 (JWT 검증 + checkout 생성)
- ☐ `/success` 페이지
- ☐ Git push → Vercel 배포
- ☐ Vercel 환경변수 입력
- ☐ Supabase / Google에 배포 도메인 추가
- ☐ Customer Portal 페이지 (Billing)
- ☐ Polar Webhook 엔드포인트 등록 + Secret 저장
- ☐ `/api/webhooks/polar` 구현
- ☐ `POLAR_WEBHOOK_SECRET` Vercel에도 추가 + 재배포
- ☐ 배포 도메인에서 Sandbox 결제 테스트

8장. 구독 설계 — 플랜과 사용량 관리

8-1. 플랜 구성

	Free	Pro	Unlimited
가격	\$0	\$9.99/월	\$29.99/월
월 호출	10회	100회	무제한
한도 초과	402 차단	402 차단	—
추천 대상	체험용	일상 사용자	파워 유저

8-2. 구독 라이프사이클 (전체 흐름)



8-3. Webhook 이벤트 종류 — canceled vs revoked 구분 필수

이벤트	의미	처리
<code>checkout.completed</code>	결제 완료	구독 생성
<code>subscription.created</code>	구독 생성	plan 기록
<code>subscription.active</code>	활성 상태 확인	status = active
<code>subscription.updated</code>	플랜 변경	plan 업데이트
<code>subscription.canceled</code>	취소 예약	기간 끝까지 유지
<code>subscription.revoked</code>	즉시 해지	바로 Free 전환
<code>subscription.past_due</code>	결제 실패	재시도 또는 경고

canceled vs revoked 차이

subscription.canceled (취소 예약)

오늘 취소 | 이번 달 남은 기간 | 다음 달부터 Free

→ 환불 없이 기간 끝까지 정상 사용

subscription.revoked (즉시 해지)

오늘 해지 | 바로 Free로

→ 환불·부정결제·약관 위반 등

8-4. DB 스키마 — subscriptions 테이블

```
create table public.subscriptions (
  user_id uuid primary key references auth.users(id) on delete cascade,
  plan text not null check (plan in ('free', 'pro', 'unlimited')),
  status text not null,
  polar_subscription_id text unique,
  polar_customer_id text,
  current_period_end timestampz,
  cancel_at_period_end boolean default false,
  updated_at timestampz default now()
);

alter table public.subscriptions enable row level security;

create policy "subscriptions_owner_read" on public.subscriptions
  for select using (auth.uid() = user_id);
```

쓰기 정책이 없는 이유

- 사용자가 플랜을 임의로 바꾸면 안 됨
- 웹훅(서버)이 **Service Role Key**로 쓰기 → RLS 우회
- 사용자는 읽기만 가능

8-5. 사용량 추적 로직

```
// app/lib/usage.ts 핵심
const LIMITS = { free: 10, pro: 100, unlimited: null };

async function getUsageFromToken(accessToken: string) {
  // 1. JWT → user 확인
  const { user } = await supabase.auth.getUser(accessToken);

  // 2. subscriptions 테이블에서 현재 플랜 조회
  const { data: sub } = await supabase
    .from("subscriptions")
    .select("plan, status")
    .eq("user_id", user.id)
    .maybeSingle();

  const plan = sub?.status === "active" ? sub.plan : "free";
  const limit = LIMITS[plan];

  // 3. 이번 달 user 메시지 수 카운트
  const { count } = await supabase
    .from("messages")
    .select("id", { count: "exact", head: true })
    .eq("role", "user")
    .gte("created_at", startOfMonthUTC())
    .in("conversation_id", userConversationIds);

  // 4. 한도 비교
  return {
    plan,
    used: count ?? 0,
    limit,
    allowed: limit === null || count < limit
  };
}
```

8-6. `/api/chat` 에서 제한 적용

```
// chat API 앞부분
const usage = await getUsageFromToken(accessToken);
if (!usage.allowed) {
  return Response.json(
    { error: "한도 초과", code: "limit_exceeded" },
    { status: 402 } // 402 Payment Required
  );
}
// 여기서 통과하면 실제 Gemini 호출
```

HTTP 상태 코드

코드	의미
200	성공
401	인증 필요
402	결제 필요 (한도 초과)
403	권한 없음
500	서버 에러

8-7. UX — 사용량 표시

대시보드 상단:

- free · 3 / 10 ← 정상
- free · 8 / 10 ← 80% 경고
- free · 10 / 10 ← 초과 → 업그레이드 모달

✓ 구독 설계 체크리스트

- ☐ `subscriptions` 테이블 + RLS 읽기 정책
- ☐ 회원가입 시 Free 자동 부여 (또는 "구독 없음 = Free"로 간주)
- ☐ Webhook 핸들러 확장 (최소 5개 이벤트)
- ☐ 웹훅 서명 검증
- ☐ `/api/chat` 에 Bearer JWT + 사용량 체크
- ☐ 한도 초과 시 402 반환
- ☐ 대시보드 사용량 표시
- ☐ 80% 경고 배너
- ☐ 100% 업그레이드 CTA 모달

9장. 테스트

9-1. 테스트는 배포된 도메인에서

로컬 vs 배포 도메인

항목	로컬 (localhost)	배포 (Vercel)
로그인	작동	작동
AI 채팅	작동	작동
결제 리다이렉트	작동	작동
Webhook 수신	✕ 외부 접근 불가	✔ 가능

→ 결제·Webhook 테스트는 반드시 배포 후

9-2. MVP 테스트 매트릭스

#	카테고리	케이스	확인 포인트
1	인증	구글 로그인	대시보드 진입
2	인증	로그아웃 후 대시보드 URL 직접 입력	<code>/login</code> 으로 리다이렉트
3	AI	메시지 전송	스트리밍 답변
4	DB	페이지 새로고침	히스토리 로드
5	RLS	다른 계정에서 A 유저 대화 조회	0건 반환
6	구독	신규 유저 플랜	plan = free, 10회 한도
7	구독	Free → Pro 결제	Webhook → plan = pro
8	한도	Free 11회 시도	402 에러
9	UI	사용량 프로그레스바	정확한 수치
10	Portal	결제 수단 변경	Polar 포털 연결

10장. 암호화 — 유저 정보 보호

배포 후 필수. DB가 털려도 정보를 못 읽게.

10-1. 왜 암호화?

시나리오

해커가 Supabase DB 털어봤다:

RLS 없이 저장:

email: hong@gmail.com ← 그대로 읽힘. 스텝 공격 받음 ✕

암호화해서 저장:

email_encrypted: a3f2b:x9z:2b3c... ← 해커가 봐도 못 모름 ✔

10-2. AES-256-GCM — 암호화 알고리즘

비유: 열쇠로 잠그는 비밀 상자.



특징

항목	설명
AES	Advanced Encryption Standard (미국 정부 표준)
256	256비트 키 길이 (매우 안전)
GCM	Galois/Counter Mode — 무결성 검증 포함
대칭키	암호화·복호화에 같은 키 사용
별명	"군사 등급 암호화"

저장 포맷

iv:authTag:encryptedData

요소	역할	비유
IV (Initialization Vector)	매번 다른 랜덤 16바이트	같은 상자라도 열쇠 구멍 위치 달라짐
authTag	위조 감지 태그	"누가 상자 흔들었는지" 알 수 있는 봉인
encryptedData	실제 암호문	상자 내용물

10-3. 해시 vs 암호화 — 헷갈리지 마세요

항목	암호화	해시
방향	양방향 (원문 복원 가능)	단방향 (원문 복원 불가)
용도	데이터 보호 + 다시 읽기	검증·검색·비밀번호
알고리즘	AES-256-GCM	SHA-256, HMAC-SHA256
결과	복호화 시 원문	같은 입력 → 항상 같은 결과

왜 둘 다 써야 하나?

- 이메일을 AES 암호화 → 매번 IV 랜덤 → **같은 이메일도 암호문이 달라짐**

- 결과: `WHERE email = 'a@b.com'` 같은 검색이 불가능

- 해결: `email_hash` 컬럼에 HMAC-SHA256(email) 저장 → 검색은 이 컬럼으로

```
저장:
email      = AES(hong@gmail.com) = "a3f2b:x9z:..." (매번 다름)
email_hash = HMAC(hong@gmail.com) = "b7f8e9..." (항상 같음)
```

```
검색:
WHERE email_hash = HMAC(hong@gmail.com)
→ 해시 일치하는 row 찾을 → AES 복호화
```

10-4. 키 2개 — ENCRYPTION_KEY, HASH_KEY

```
ENCRYPTION_KEY=64자리_hex ← AES 암호화용 (32바이트)
HASH_KEY=64자리_hex       ← HMAC 해시용 (32바이트)
```

키 생성 예시

```
const crypto = require("crypto");
console.log(crypto.randomBytes(32).toString("hex"));
// → 7a3b9c2e1f4d... 이런 거 64자리
```

10-5. 절대 규칙 — 키 관리

규칙	이유
키는 서버 전용	클라이언트 노출 → 모든 데이터 해독됨
<code>.env.local</code> + Vercel 양쪽 동기화	한쪽만 있으면 반쪽 서비스 장애
키 변경 금지	바꾸면 기존 암호문 전부 해독 불가 = 데이터 유실
git 커밋 금지	한 번 푸시되면 히스토리 영구
분실 = 복구 불가	1Password 같은 곳에 별도 백업

10-6. 구현 요소 5가지

1. `lib/encryption.ts` — 유틸 함수

```
encrypt(text) → "iv:authTag:cipher" decrypt(stored) → 원문 hashForLookup(text) → HMAC 해시
```

2. `scripts/generate-keys.ts` — 키 생성 스크립트

3. DB 스키마 변경

```
sql alter table profiles add column email_hash text; create index profiles_email_hash_idx on profiles(email_hash);
```

4. 마이그레이션 스크립트 — 기존 평문 데이터 암호화

5. 앱 플로우 수정

- 저장 시: encrypt + hash
- 조회 시: decrypt
- 검색 시: WHERE email_hash = ...

10-7. 검증 방법

Supabase 대시보드 → Table Editor:

```
Before (암호화 전):  
email: hong@gmail.com  
  
After (암호화 후):  
email: a3f2b:x9z:2b3c4d5e6f...  
email_hash: b7f8e9a0...
```

앱에서는 여전히 `hong@gmail.com` 로 보임 (서버가 복호화해서 전달).

✓ 암호화 체크리스트

- ☐ `lib/encryption.ts` 3개 함수
- ☐ 키 생성 스크립트
- ☐ ENCRYPTION_KEY, HASH_KEY → `.env.local`
- ☐ 해시 컬럼 + 인덱스 생성
- ☐ 회원가입·조회 플로우 암호화 적용
- ☐ 기존 데이터 마이그레이션 실행
- ☐ RLS 정책 재검토
- ☐ Vercel 환경변수에 키 추가
- ☐ 암호화된 DB 확인
- ☐ 앱 정상 동작 확인
- ☐ 키 안전한 곳에 백업

11장. 환경변수 총정리

11-1. 전체 목록

```
# ----- Supabase -----
NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=[publishable key]
SUPABASE_SECRET_KEY=[secret key]

# ----- Gemini -----
GOOGLE_API_KEY=AIZA...

# ----- Polar -----
POLAR_ACCESS_TOKEN=polar_oat_...
POLAR_WEBHOOK_SECRET=...
POLAR_PRODUCT_ID_PRO=...
POLAR_PRODUCT_ID_UNLIMITED=...
POLAR_SERVER=sandbox

# ----- 암호화 (선택) -----
ENCRYPTION_KEY=[64자리 hex]
HASH_KEY=[64자리 hex]
```

11-2. 접두사 규칙 요약

NEXT_PUBLIC_... 접두사 → 공개 OK 값만	클라이언트에 노출됨
(접두사 없음) → 시크릿, API 키, secret	서버 전용

11-3. 2곳 동기화 – 가장 흔한 실수

로컬 개발 배포 운영

.env.local → Vercel Environment Variables
(둘 다 똑같이 유지)

자주 하는 실수

- ✕ 로컬에만 추가 → 배포 사이트에서 undefined
- ✕ 환경변수 수정 후 재배포 안 함 → 구 값 사용
- ✕ 시크릿에 `NEXT_PUBLIC_` 붙임 → 브라우저에 노출

12장. AI 프롬프트 작성법

12-1. 좋은 프롬프트 공식

[무엇을 만들지]
 + [왜 / 맥락]
 + [요구사항 나열]
 + [사용할 도구·제약]

12-2. 좋은 예시 vs 나쁜 예시

✗ 나쁜 예시 — 모호함

"로그인 만들어줘"

✓ 좋은 예시 — 명확

"Supabase Google OAuth 로그인을 구현해줘.
 요구사항:
 1. 로그인 페이지에 구글 버튼
 2. AuthContext로 전역 상태 관리
 3. 미로그인 시 /dashboard → /login 리다이렉트
 4. 로그아웃 기능
 환경변수는 .env.local에:
 NEXT_PUBLIC_SUPABASE_URL
 NEXT_PUBLIC_SUPABASE_ANON_KEY"

12-3. 디버깅 프롬프트 공식

[증상 + 에러 메시지 전체 복사]
 + [언제 발생했는지]
 + [최근 변경사항]

예시

"결제 후 /success 페이지에서 이 에러가 떠:

```
TypeError: Cannot read properties of undefined (reading 'access_token')
    at handleUpgrade (pricing/page.tsx:45)
```

최근에 useAuth 훅에서 session 상태 로직을 바꿨어.
 원인 분석하고 고쳐줘."

12-4. UI 프롬프트 공식

[레퍼런스 이미지 or 링크]
 + [목적]
 + [다크/라이트, 반응형 여부]
 + [세부 텍스트·CTA]

예시

"이 레퍼런스 스크린샷처럼 pricing 페이지 만들어줘.
목적: Gemini 래퍼 SaaS의 요금제
다크 테마, 반응형 (모바일 1열, 데스크탑 3열)
3개 카드: Free / Pro / Unlimited
Pro 카드는 약간 강조 (추천 배지)
각 카드 하단 CTA: Free=현재 플랜 (비활성), Pro/Unlimited='업그레이드' "

13장. 자주 겪는 에러 사전

증상	원인	해결
<code>npm: command not found</code>	Node.js 미설치	nodejs.org에서 LTS 설치 + 터미널 재시작
로그인 후 <code>redirect_uri_mismatch</code>	Google Console URI 불일치	Supabase callback URL 복사해서 Google에 등록
<code>Invalid API key</code>	환경변수 오타·공백	키 재확인, 서버 재시작 (<code>npm run dev</code> 재실행)
<code>new row violates row-level security policy</code>	RLS 정책 누락	에이전트에게 "RLS 정책 SQL 만들어줘"
Webhook 403	Secret 불일치	Polar 재발급 → Vercel 환경변수 → 재배포
배포 후 env undefined	Vercel에 env 추가 안 함	Settings → Environment Variables + Redeploy
Vercel <code>email not matched to GitHub</code>	git user.email이 GitHub 미등록	GitHub Settings → Emails에 추가
<code>localhost:3000</code> 안 열림	dev 서버 미실행	<code>npm run dev</code>
TypeScript 에러	타입 불일치	<code>npx tsc --noEmit</code> 으로 위치 확인 후 AI에 전달
Vercel 빌드 실패	env 누락 or TS 에러	빌드 로그 그대로 AI에 복붙
결제 후 플랜 안 바뀜	Webhook 미수신	Polar Dashboard → Webhooks → 로그 확인

해결의 80%: 에러 메시지를 그대로 복사해서 AI에 전달.

14장. 용어 사전

A. 기본 웹 개발 용어

용어	정의
HTTP	웹에서 서버·클라이언트가 대화하는 규약
HTTPS	HTTP + 암호화 (공격자가 엿볼 수 없음)
URL	인터넷 주소 (<code>https://도메인/경로?파라미터</code>)
Domain	사이트 주소의 이름 부분 (<code>google.com</code>)
DNS	도메인 이름 → IP 주소 변환
IP 주소	인터넷상 컴퓨터의 번호
Port	한 컴퓨터에서 여러 서비스 구분용 번호 (3000, 80, 443)

B. 개발 일반

용어	정의
SaaS	Software as a Service — 월 구독형 소프트웨어
BaaS	Backend as a Service — 백엔드를 통째로 제공 (Supabase)
MVP	Minimum Viable Product — 최소 기능 제품
프론트엔드	사용자가 보는 화면
백엔드	사용자가 안 보는 서버·DB
클라이언트	서비스 사용자 쪽 (브라우저)
서버	서비스 제공자 쪽

C. API & 통신

용어	정의
API	프로그램 간 통신 규격
REST API	HTTP 메서드로 주고받는 API
Endpoint	API의 URL 경로 (<code>/api/chat</code>)
Request / Response	요청 / 응답
JSON	데이터 표현 형식 (<code>{"key": "value"}</code>)
GET / POST / PUT / DELETE	HTTP 메서드 (조회/생성/수정/삭제)

D. Next.js & React

용어	정의
React	UI 라이브러리 (페이스북 개발)
Next.js	React 기반 풀스택 프레임워크
Component	재사용 가능한 UI 조각
Hook	React 함수 (예: <code>useState</code> , <code>useEffect</code>)
Server Component	서버에서 렌더링되는 컴포넌트
Client Component	브라우저에서 실행되는 컴포넌트 (<code>"use client"</code>)
Route Handler	API 엔드포인트 파일 (<code>route.ts</code>)
App Router	Next.js 13+의 라우팅 방식 (파일=URL)

E. 인증 & 보안

용어	정의
OAuth	제3자 서비스로 로그인하는 표준
JWT	JSON Web Token — 서명된 인증 토큰
Session	로그인 상태 유지 데이터
Cookie	브라우저에 저장되는 작은 데이터
CORS	다른 도메인 간 요청 허용 규칙
CSRF / XSS	웹 공격의 종류들
RLS	Row Level Security — 행 단위 접근 제어

F. 데이터베이스

용어	정의
SQL	데이터베이스 표준 언어
PostgreSQL	오픈소스 관계형 DB (Supabase 기반)
Table / Row / Column	테이블 / 행 / 열
Primary Key (PK)	각 행을 고유 식별하는 키
Foreign Key (FK)	다른 테이블 참조
INDEX	조회 속도 향상 구조
Migration	스키마 변경 스크립트
ORM	객체지향 코드로 DB 다루기 (Prisma 등)

G. 결제 & 구독

용어	정의
Checkout Session	결제 페이지 세션
Webhook	서버→서버 이벤트 알림
Subscription	반복 결제 (월/연)
MRR	Monthly Recurring Revenue
Churn	구독 취소율

H. AI & LLM

용어	정의
LLM	Large Language Model (GPT, Gemini 등)
Token	LLM이 처리하는 텍스트 단위 (대략 4글자=1토큰)
Streaming	응답을 조각 단위로 전송
Prompt	AI에게 주는 지시문
MCP	Model Context Protocol

I. 암호학

용어	정의
AES	Advanced Encryption Standard — 표준 암호화
HMAC	키 기반 해시 (검색·서명용)
SHA-256	해시 알고리즘
IV	Initialization Vector — 암호화 시 랜덤값
대칭키 / 비대칭키	같은 키 / 다른 키(공개+비밀)

J. 운영 & 배포

용어	정의
CI/CD	Continuous Integration / Deployment
Deployment	서비스 배포
Environment Variable	환경에 따라 다른 설정값
Serverless	서버 직접 관리 없이 함수 단위 실행
Edge Function	사용자 가까운 곳에서 실행되는 함수

15장. 다음 단계 학습 로드맵

15-1. 깊이 학습 (이미 쓴 기술 더 파보기)

주제	왜 필요	추천 자료
React 훅 심화	복잡한 상태 관리	React 공식 문서
Server vs Client Component	경계 판단	Next.js 공식 문서
SQL 기초	JOIN, GROUP BY, 인덱스 최적화	SQLBolt 인터랙티브 튜토리얼
Zod 검증	API 입력 안전하게	Zod 공식 문서

15-2. 넓이 확장 (인접 기술)

주제	역할	우선순위
Stripe	Polar 대신 더 성숙한 결제	★★
Prisma / Drizzle	타입 안전 ORM	★★
Redis / Upstash	캐싱 · rate limiting	★
Vercel AI SDK	AI 기능 고도화	★★★
Vitest + Playwright	자동 테스트	★★
Sentry	에러 모니터링	★★
PostHog	유저 행동 분석	★★
Resend	이메일 발송	★★

15-3. 비즈니스 관점

- **유입 분석:** UTM, SEO, 랜딩 A/B 테스트
- **결제 메트릭:** MRR, Churn, LTV 추적
- **고객 지원:** Intercom, Crisp 같은 채팅 툴

16장. 최종 정리

머릿속에 남겨야 할 3가지

① "서버 vs 클라이언트" 경계를 항상 의식



`NEXT_PUBLIC_` 접두사 = 공개됨. 시크릿에는 절대 붙이지 않기.

② 데이터 흐름을 그림으로 그릴 수 있어야

이 문서의 다이어그램들 (2-1, 5-5, 7-4 등)을 기억해두세요.
새 프로젝트 시작할 때 이 그림만 그려도 구조가 잡힙니다.

③ 환경변수·시크릿 관리가 디버깅의 80%

"로컬 OK, 배포 NG" → Vercel env 확인
"어제 OK, 오늘 NG" → 최근 변경한 키 확인
"API 호출 실패" → 환경변수 오타·공백 확인

벽은 생각보다 낮습니다

벽	본질	해결
WALL 1: 시작	코드 작성·저장	Antigravity + Next.js + GitHub
WALL 2: 인증	누가 쓰는지	Supabase + Google OAuth
WALL 3: 결제	돈 받기	Polar + Vercel + Webhook

다음 프로젝트에선...

이 문서는 지도입니다.

새 SaaS를 만들 때, 여기 배운 구조

"Supabase 인증 → AI API → Polar 결제 → Vercel 배포"

를 그대로 적용하고 도메인만 바꾸면 됩니다.

한 번 넘어본 벽은 다시 만나면 더 낮아져 있습니다.

작성 메모

- 이 문서는 마크다운(`.md`) 파일입니다.
- PDF로 변환: VS Code + "Markdown PDF" 확장 / Typora / Notion에 붙여넣기
- 인쇄용: 브라우저로 열어서 Ctrl+P → "PDF로 저장"
- 수정: 일반 텍스트처럼 편집 가능

끝.